

Objektorienterad programmering i C#

C1OJ1B I12V6 VT2026

2026-05-13



HÖGSKOLAN
I BORÅS

Laboration 1: Avancerad objektorienterad
programmeringsuppgift - Design reflektion

Grupp 7:

Helin Akgül och Alisar Sido

Innehållsförteckning

Innehållsförteckning.....	2
Inledning.....	3
Eventuellt info om LocalDB.....	3
Klassdiagram.....	4
Kort arkitekturbeskrivning.....	5
Sekvensdiagram.....	6
Use Cases.....	7
Case 1.....	7
Case 2.....	10
Case 3.....	12
Reflektionsdel.....	13

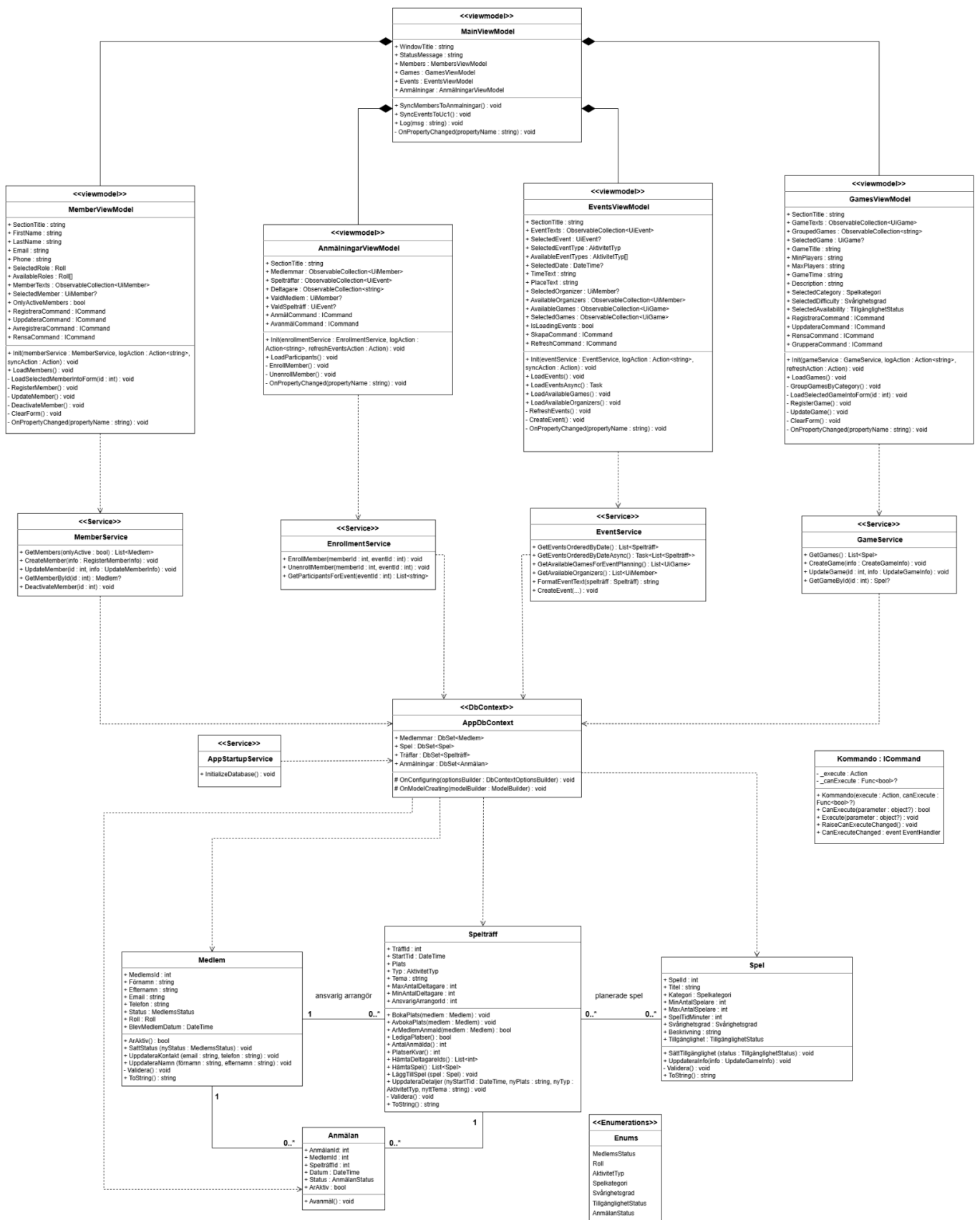
Inledning

Detta dokument beskriver vidare utvecklingen av vårt system för verksamheten från laboration 1. Arbetet byggdes vidare efter den domänmodell som togs fram i förra projektet med centrala objekt som medlemmar, spel, spelträffar och anmälningar. I laboration 2 har fokus legat på att vidareutveckla lösningen till ett mer komplett objektorienterat system som passar verksamhetens behov med WPF gränssnitt, MVVM-struktur, databinding, commands, Entity Framework och persistent lagring.

Eventuellt info om LocalDB

Applikationen använder Entity Framework mot en SQL Server-kompatibel databas, där vi har valt SQL Server LocalDB som lokal databasmotor. I koden används LocalDB-instansen (localdb)\MSSQLLocalDB, och databasen skapas automatiskt vid uppstart genom EnsureCreated(). I den här lösningen används inga migrations, utan databasen byggs upp direkt från modellklasserna. Om datamodellen ändras på ett sätt som kräver nya tabeller eller ändrad struktur behöver databasen tas bort och skapas om på nytt. Den här lösningen uppfyller laborationens krav på persistent lagring med Entity Framework och en tydlig strategi för hur databasen skapas. Vi använder även demonstrationsdata, så att centrala funktioner kan visas direkt utan omfattande manuell förberedelse. Detta passar laborationens krav på att lösningen ska innehålla rimlig demonstrationsdata för redovisning.

Klassdiagram



Kort arkitekturbeskrivning

Applikationen är byggd som en WPF-applikation enligt MVVM-principen. Views ansvarar för presentationen i XAML, medan ViewModels ansvarar för tillstånd, databindning, valt objekt, formulärfält och commands. Det ligger i linje med laborationens krav att presentationslagret ska organiseras enligt MVVM, att databindning ska användas korrekt och att centrala knapptryckningar ska hanteras via commands i en ViewModel.

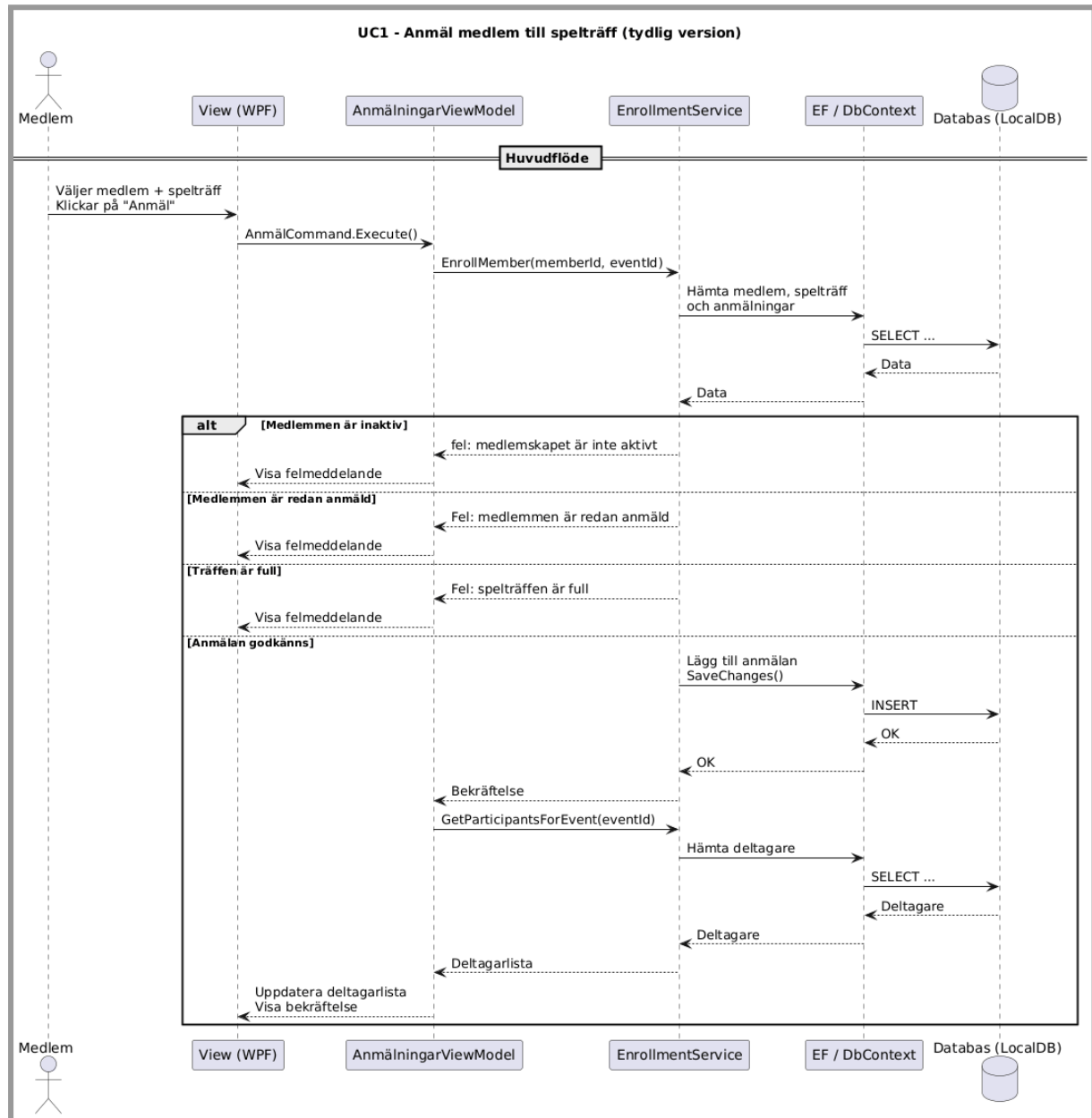
Lösningen är uppdelad i flera tydliga lager. Domänlagret innehåller verksamhetens centrala klasser och regler, till exempel medlemmar, spel, spelträffar och anmälningar. Services-lagret innehåller applikationslogik och kapslar in arbetsflöden som registrering, uppdatering, skapande av spelträffar samt anmälan och avanmälan. Detta följer laborationens krav på tydlig separation mellan presentation, applikationslogik och dataåtkomst, samt att lösningen ska vara begripligt organiserad.

Data Åtkomsten är kapslad i ett särskilt datalager med Entity Framework och en ApplicationDbContext. Databasen lagras i SQL Server LocalDB, vilket är ett rekommenderat alternativ i laborationen. Data skapas och sparas persistent i databasen och finns kvar efter omstart. Databasen skapas automatiskt vid uppstart med en tydlig startstrategi baserad på EnsureCreated(). Detta motsvarar laborationens krav på persistent lagring med Entity Framework och att ni ska kunna förklara hur databasen skapas eller uppdateras.

Applikationen använder dessutom demonstrationsdata som seedas vid uppstart för att centrala funktioner och arbetsflöden ska kunna visas direkt i redovisningen utan omfattande manuell förberedelse. Detta är i linje med instruktionens krav på rimlig demonstrationsdata.

Sammanfattningsvis bygger arkitekturen på en tydlig uppdelning där WPF/XAML används för presentation, ViewModels för presentationslogik och tillstånd, services för applikationslogik och verksamhetsflöden, samt Entity Framework/LocalDB för dataåtkomst och persistent lagring. Denna struktur gör det lättare att följa ansvarsfördelningen i systemet och ligger nära laborationens mål om en välstrukturerad applikation ovanpå en objektorienterad kärna.

Sekvensdiagram



Use Cases

Case 1

Namn: Anmäl medlem till spelträff

Primär aktör: Medlem

Sekundär aktör: Arrangör eller administratör

Mål

Målet är att en medlem ska kunna anmälas till en kommande spelträff via applikationens GUI och att systemet ska:

- kontrollera att verksamhetens regler följs,
- spara anmälan persistent i databasen,
- uppdatera deltagarlistan,
- och ge tydlig återkoppling till användaren.

Förutsättningar

1. Det finns minst en registrerad spelträff i systemet med datum, tid och plats.
2. Det finns minst en registrerad medlem i systemet.
3. Medlemmen har ett aktivt medlemskap. Eftersom applikationen ska hantera verksamhetsregler och ogiltiga operationer tydligt är detta en relevant regel att kontrollera före anmälan.
4. Spelträffen är inte avslutad och ska vara möjlig att anmäla sig till.
5. Applikationen är igång och användaren befinner sig i den del av gränssnittet där medlemmar och spelträffar kan väljas, vilket motsvarar kravet att arbetsflöden ska kunna genomföras via WPF-gränssnittet.

Huvudflöde

1. Användaren öppnar vyn för anmälningar i applikationen. Arbetsflödet ska kunna genomföras i det grafiska gränssnittet enligt laborationens krav på en fungerande WPF-applikation.

2. Systemet visar en lista med medlemmar och en lista med kommande spelträffar. Instruktionen kräver att centrala objekt ska kunna visas i GUI och att listor/valt objekt ska bindas korrekt.
3. Användaren väljer en medlem i medlemslistan. Databindning mot valt objekt är en uttrycklig del av laborationens tekniska krav.
4. Användaren väljer en spelträff i listan över kommande spelträffar. Även detta ingår i den användarsituation där objekt väljs, kopplas samman och sparas.
5. Systemet visar information om den valda träffen, till exempel deltagare och/eller aktuell status. Detta stödjer kravet att användaren ska kunna visa centrala objekt och detaljer i applikationen.
6. Användaren klickar på Anmäl. Eftersom detta är ett centralt knapptryck som förändrar data ska det hanteras via command i ViewModel, enligt labbens MVVM-krav.
7. Systemet kontrollerar att:
 - a. medlemmen har aktivt medlemskap,
 - b. spelträffen inte är fullbokad,
 - c. medlemmen inte redan är anmäld,
 - d. och att anmälan i övrigt är tillåten enligt verksamhets reglerna.
8. Om kontrollerna godkänns skapar systemet en anmälan och sparar den persistent i databasen via Entity Framework.
9. Systemet uppdaterar deltagarlistan för den valda spelträffen och visar att medlemmen nu är anmäld.
10. Systemet visar ett tydligt bekräftelsemeddelande till användaren, till exempel att medlemmen har anmälts till spelträffen.

Efter Villkor

Vid lyckat genomförande

- Det finns en sparad anmälan kopplad till medlemmen och spelträffen i databasen.
- Deltagarlistan för spelträffen är uppdaterad i GUI:t.
- Anmälan finns kvar även efter att applikationen stängts och startats igen, eftersom data ska vara persistent.

Vid misslyckat genomförande

- Ingen anmälan sparas i databasen.
- Systemet visar ett tydligt felmeddelande till användaren.

Alternativa flöden

A1. Träffen är full

Vid steg 7 upptäcker systemet att maxantal deltagare redan har uppnåtts.

Systemet stoppar anmälan, sparar inget i databasen och visar ett tydligt felmeddelande, till exempel: "Spelträffen är fullbokad."

A2. Medlemmen är redan anmäld

Vid steg 7 upptäcker systemet att medlemmen redan har en aktiv anmälan till samma spelträff.

Systemet stoppar dubbelanmälan, sparar inget nytt och visar ett tydligt felmeddelande, till exempel: "Medlemmen är redan anmäld till spelträffen."

A3. Medlemmen är inte aktiv

Vid steg 7 upptäcker systemet att medlemmen inte har aktiv status.

Systemet stoppar anmälan och visar ett tydligt felmeddelande, till exempel: "Endast aktiva medlemmar får anmälas till en spelträff."

A4. Ingen medlem eller ingen spelträff är vald

Vid steg 3 eller 4 har användaren inte gjort ett giltigt val.

Systemet genomför inte anmälan och visar ett tydligt meddelande, till exempel: "Välj både en medlem och en spelträff."

A5. Avamälan

Användaren väljer Avamäl för en medlem som redan är anmäld till en spelträff.

Systemet tar då bort anmälan. Efter det uppdaterar systemet deltagarlistan och visar den nya statusen för spelträffen.

Case 2

Namn: Skapa och hantera en ny spelträff.

Primär aktör: Administratör eller arrangör.

Mål

Målet är att administratören eller arrangören ska kunna skapa en ny spelträff i applikationens GUI genom att ange nödvändig information, så att träffen sparas persistent i databasen och därefter kan visas i listan över kommande aktiviteter och bli möjlig att anmäla sig till.

Förutsättningar

1. Spelträff skapas och användaren trycker in arrangören.
2. Systemet är igång och användaren befinner sig i den del av applikationen där spelträffar kan skapas.
3. Det finns tillräcklig grunddata i systemet, till exempel registrerade medlemmar/arrangörer och eventuellt spel som kan kopplas till spelträffen.

Huvudflöde

1. Användaren öppnar vyn för spelträffar i applikationen.
2. Systemet visar en lista över befintliga eller kommande spelträffar samt ett formulär där en ny spelträff kan skapas.
3. Användaren väljer funktionen Skapa ny spelträff och anger grundläggande information, till exempel: datum, tid, plats, aktivitetstyp, eventuell ansvarig arrangör, och eventuellt kopplade spel eller tema.
4. Användaren klickar på Skapa spelträff.
5. Systemet validerar att obligatoriska uppgifter är ifyllda och att de angivna värdena är rimliga, till exempel att datum, tid och plats finns angivna.

6. Om uppgifterna är giltiga skapar systemet en ny spelträff i domänmodellen och sparar den persistent i databasen via Entity Framework.
7. Systemet uppdaterar listan över kommande spelträffar i GUI:t så att den nya spelträffen blir synlig direkt.
8. Systemet visar ett tydligt meddelande om att spelträffen har skapats.

Efter Villkor

Vid lyckat genomförande

1. Den nya spelträffen finns sparad i databasen och kan läsas tillbaka efter omstart.
2. Spelträffen visas i gränssnittet bland kommande aktiviteter.
3. Spelträffen kan därefter användas i andra arbetsflöden, till exempel anmälan av medlemmar.

Vid misslyckat genomförande

1. Ingen spelträff sparas i databasen.
2. Användaren får tydlig återkoppling om varför skapandet inte kunde genomföras.

Alternativa flöden

A1. Obligatorisk information saknas

Vid steg 5 upptäcker systemet att nödvändig information saknas, till exempel datum, tid eller plats. Systemet skapar inte spelträffen och visar tydligt vilka uppgifter som måste kompletteras.

A2. Ogiltiga värden anges

Vid steg 5 eller 6 upptäcker systemet orimliga eller ogiltiga värden, till exempel att maxantal deltagare är för lågt eller att tiden är i fel format. Systemet stoppar skapandet och ber användaren korrigera uppgifterna.

Case 3

Namn: Registrera och uppdatera medlemmar.

Primär aktör: Användaren .

Mål

Målet är att Användaren ska kunna registrera en ny medlem och uppdatera information om befintliga medlemmar i systemet. Medlems informationen ska sparas persistent i databasen och användas av övriga centrala arbetsflöden, till exempel anmälan till spelträffar.

Förutsättningar

1. Systemet är i drift och medlemshantering är tillgänglig via GUI:t.
2. Databasen är tillgänglig så att medlems informationen kan sparas persistent med Entity Framework.

Huvudflöde

1. Användaren öppnar vyn för medlemshantering i applikationen.
2. Systemet visar en lista över befintliga medlemmar samt ett formulär för att registrera eller uppdatera medlemsinformation.
3. Användaren väljer att registrera en ny medlem eller markerar en befintlig medlem för uppdatering.
4. Användaren anger eller ändrar medlemsuppgifter, till exempel: förnamn, efternamn, e-post, telefon, roll, och eventuell status.
5. Användaren klickar på Registrera eller Uppdatera.
6. Systemet kontrollerar att obligatoriska uppgifter är korrekt ifyllda och att uppgifterna följer verksamhetens regler, till exempel att samma e-postadress inte används dubbelt.
7. Om uppgifterna är giltiga skapar eller uppdaterar systemet medlems objektet och sparar ändringen persistent i databasen
8. Systemet uppdaterar medlemslistan i GUI:t så att den nya eller uppdaterade medlemmen visas direkt.
9. Systemet visar en tydlig bekräftelse till personen.

Efter Villkor

Vid lyckat genomförande

1. Den nya eller uppdaterade medlemmen finns sparad i databasen och kan läsas tillbaka efter omstart.
2. Medlemslistan i GUI:t är uppdaterad.
3. Om medlemmen är aktiv kan medlemmen därefter användas i andra centrala arbetsflöden, till exempel anmälan till spelträff.

Vid misslyckat genomförande

1. Ingen ny medlem skapas och Användaren får tydlig information om vad som behöver rättas.
2. ingen ogiltig uppdatering sparas.

Alternativa flöden

A1. Obligatoriska uppgifter saknas

Vid steg 6 upptäcker systemet att en eller flera obligatoriska uppgifter saknas, till exempel namn eller kontaktuppgifter. Systemet skapar eller uppdaterar inte medlemmen och visar tydligt vilka uppgifter som måste kompletteras.

A2. Medlem finns redan registrerad

Vid steg 6 upptäcker systemet att en medlem med samma identifierande uppgifter redan finns registrerad, till exempel samma e-postadress. Systemet stoppar registreringen och informerar användaren om att medlemmen redan finns.

A3. Befintlig medlem uppdateras

I stället för att registrera en ny medlem väljer användaren en befintlig medlem i listan och uppdaterar uppgifter som e-post, telefon eller namn. Systemet sparar den uppdaterade informationen och visar den nya versionen i medlemslistan.

Reflektionsdel

I laboration 2 har vi vidareutvecklat den objektorienterade lösningen från förra arbetet till ett mer komplett system. Utgångspunkten har varit samma verksamhetsbehov och central

domänlogik med klasserna *Medlem*, *Spel*, *Spelträff* och *anmälan*. Vi har förbättrat applikationen genom presentationslager, databindning, commands, Entity Framework och med persistent lagring. Vårt designval bygger på grundläggande principer inom objektorientering, där en viktig del är att dela upp ansvaret och lösningen i mindre delar. Domänklasserna ansvarar för verksamhetens centrala begrepp och regler, bland annat medlemsstatus, speldetaljer, anmälningar och begränsningar kopplade till spelträffar. *ViewModels* ansvarar för det som hör till gränssnittet, exempelvis valda objekt, formulärfält, listor och commands. *Services* används som mellanlager för applikationslogik och dataåtkomst, däribland registrering av medlemmar, hantering av spel, skapande av spelträffar samt anmälan och avanmälan. Genom denna uppdelning separeras vyerna från den direkta dataåtkomsten vilket bidrar till en tydligare ansvarsfördelning i systemet och gör arkitekturen mer strukturerad.

Vi har tillämpat MVVM-principen genom att låta *MainWindow.xaml* ansvara för presentationen, medan logiken för användarflöden hanteras i separata *ViewModels*. Centrala knapptryckningar hanteras via kommandon i respektive *ViewModel* som därmed minskar behovet av logik i code-behind. Code-behind hålls därför mycket begränsad och används i huvudsak för att initiera fönstret samt koppla DataContext. Detta bidrar till en mer strukturerad lösning som är lättare att underhålla och vidareutveckla än om all logik hade placerats direkt i XAML-filens code-behind.

Persistent lagring har implementerats med hjälp av Entity Framework och SQL Server LocalDB. *AppDbContext* används för att hantera databaskopplingen samtidigt som services kapslar in databasoperationerna så att datahanteringen blir mer kontrollerat och separeras från övriga delar av applikationen. Databasen skapas automatiskt vid uppstart med `EnsureCreated()` och demonstrationsdata seedas så att applikationen kan visas utan omfattande manuell förberedelse. En mer avancerad framtida version skulle kunna använda migrations för att hantera databas förändringar mer kontrollerat.

Felhantering och validering har fördelats mellan flera delar av applikationen. Domänklasserna ansvarar för grundläggande verksamhetsregler, exempelvis krav på giltiga uppgifter, rimliga värden och begränsningar för spelträffar. *Services* kontrollerar arbetsflöden kopplade till databasen såsom att relevanta objekt finns, att medlemsstatus är giltig och att samma anmälan inte registreras flera gånger. *ViewModels* hanterar exceptions samt

presenterar felmeddelanden i gränssnittet och på så sätt kan vanliga användarfel hanteras utan att programmet kraschar.

Vi har även implementerat en asynkron funktion för att visa att applikationen tar hänsyn till responsivitet. Laddning av spelträffar kan ske asynkront via *EventService* och *EventsViewModel* där Entity Frameworks asynkrona metoder används för att hämta data från databasen. Funktionen är placerad i *Services* och *ViewModels* eftersom det är där data hämtas och presenteras för användaren. Detta är relevant eftersom databasanrop annars kan riskera att blockera gränssnittet om mängden data eller svarstiden ökar.

De tidigare in-memory-baserade registerklasserna har ersatts med persistent lagring samt en tydligare uppdelning mellan GUI, ViewModels, services och databas. GUI:t arbetar därför genom ViewModels och services i stället för att kommunicera direkt med samlingar eller databaskontexten. Vi har även infört request-klasser vid skapande respektive uppdatering av spel och medlemmar, för att minska långa parameterlistor samt göra metod anropen mer strukturerade.

Det finns flera möjliga förbättringar för att vidareutveckla systemet. Behörighetshanteringen skulle kunna utökas med mer avancerad logik för administratörer, arrangörer och medlemmar för att ge bättre kontroll över funktioner och åtkomst. Även hanteringen av spelens tillgänglighet kan automatiseras, exempelvis genom att ett spel markeras som reserverat när en spelträff skapas och blir tillgängligt igen när träffen genomförs eller avbokas. Dessutom skulle presentationslagret kunna göras mer modulärt genom att dela upp MainWindow.xaml i flera separata UserControls, vilket skulle förbättra kodens struktur, underhållbarhet och möjligheten att vidareutveckla applikationen.